



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e  
INTERACCIÓN HUMANO COMPUTADORA



## **REPORTE DE PRÁCTICA N° 03**

**NOMBRE COMPLETO: GARCÍA SITIO DIEGO**

**N° de Cuenta: 318120601**

**GRUPO DE LABORATORIO: 13**

**GRUPO DE TEORÍA: 06**

**SEMESTRE 2027-1**

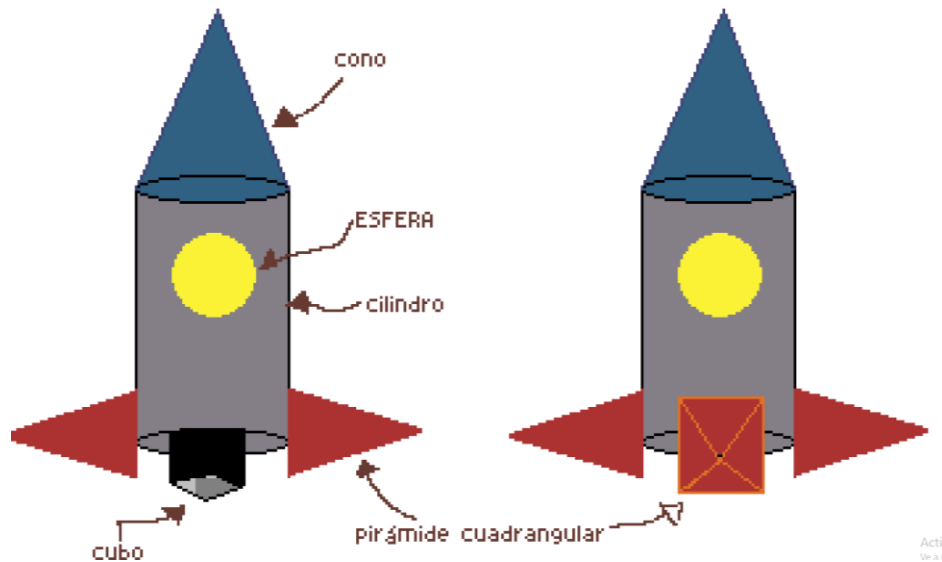
**FECHA DE ENTREGA LÍMITE: 17/09/2026**

**CALIFICACIÓN: \_\_\_\_\_**

## EJERCICIO 1

**Descripción:** Crear un cohete espacial con instancias de: pirámides, cilindros, esferas, conos, y cubos.

**Solución:** Para hacer mi cohete, realicé un boceto para tener una guía visual de cómo debía hacerlo, siendo el resultado plasmado en la ilustración 1.



*Ilustración 1: Boceto del ejercicio 1.*

Para hacer el cilindro de mi cohete, tuve que hacer el código de la ilustración 2 (siendo lo más resaltante el color y la modificación de su altura) y también aumenté las subdivisiones en el cilindro de la ilustración 3 porque me daba una figura distinta a la deseada.

```
1
2
3 // Ejercicio 1: cohete
4 glm::mat4 rocketBase = glm::translate(baseModel, glm::vec3(-3.0f, 0.0f, 0.0f));
5 // Cilindro - meshList[2]
6 shaderList[0].useShader();
7 uniformModel = shaderList[0].getModelLocation();
8 uniformProjection = shaderList[0].getProjectLocation();
9 uniformView = shaderList[0].getViewLocation();
10 uniformColor = shaderList[0].getColorLocation();
11 //
12 glm::mat4 modelCuerpo = glm::scale(rocketBase, glm::vec3(1.0f, 5.0f, 1.0f));
13 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelCuerpo));
14 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
15 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(view));
16 glm::vec3 colorGris = glm::vec3(0.5f, 0.5f, 0.5f);
17 glUniform3fv(uniformColor, 1, glm::value_ptr(colorGris));
18 meshList[2] -> RenderMeshGeometry();
19
```

*Ilustración 2: Código del cilindro.*

```

    CrearPiramideTriangular();//índice 1 en MeshList
    CrearCilindro(20, 1.0f);//índice 2 en MeshList
    CrearCono(25, 2.0f);//índice 3 en MeshList

```

*Ilustración 3: 20 subdivisiones para el cilindro.*

Para las ilustraciones 4 y 5, que corresponden al código del cono y la esfera, solo empleé las instrucciones del profesor, junto con modificaciones de su escala para encontrar una figura que me agradara. Intenté agregar más detalles de los que plasmé en mi boceto, pero realmente ninguno me gustó. Pregunté a la IA si me recomendaba otros colores a los que había planteado, pero no hubo muchas buenas sugerencias, así que dejé los que ya había decidido.

```

511 //El cono - meshList[3]
512 shaderList[2].useShader(); // Azul
513 uniformModel = shaderList[2].getModelLocation();
514 uniformProjection = shaderList[2].getProjectLocation();
515 uniformView = shaderList[2].getViewLocation();
516 //
517 glm::mat4 modelPunta = glm::translate(rocketBase, glm::vec3(0.0f, 3.25f, 0.0f));
518 modelPunta = glm::scale(modelPunta, glm::vec3(0.5f, 1.5f, 0.5f));
519 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelPunta));
520 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
521 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(view));
522 meshList[3]->RenderMeshGeometry();
523

```

*Ilustración 4: Código del cono.*

```

//Las esferas
shaderList[7].useShader(); // Amarillo
uniformModel = shaderList[7].getModelLocation();
uniformProjection = shaderList[7].getProjectLocation();
uniformView = shaderList[7].getViewLocation();
// 1
glm::mat4 modelVentana_01 = glm::translate(rocketBase, glm::vec3(0.0f, 0.5f, 1.0f)); //0.0f, 1.5f, 1.0f
modelVentana_01 = glm::scale(modelVentana_01, glm::vec3(0.4f, 0.4f, 0.4f)); //0.2f, 0.2f, 0.2f
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelVentana_01));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(view));
sp.render();
// 2
/*
glm::mat4 modelVentana_02 = glm::translate(rocketBase, glm::vec3(0.0f, 1.5f, 1.0f)); //
modelVentana_02 = glm::scale(modelVentana_02, glm::vec3(0.2f, 0.2f, 0.2f)); //0.4f, 0.4f, 0.4f
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelVentana_02));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(view));
sp.render();
*/

```

*Ilustración 5: Código de la esfera.*

Para las aletas y el motor, que corresponden a la ilustración 6 y 7, decidí retomar las variables para modificar instantáneamente varias figuras al mismo tiempo. De esta forma me ayudaba a ubicar y escalar las aletas de manera más sencilla. Como se ve en la ilustración 8, comencé a modificar todas las partes de las figuras para ajustar el cohete a mi boceto.

```

547 //Aletas - 4 Pirámides cuadrangulares
548 shaderList[6].useShader(); // Rojo
549 uniformModel = shaderList[6].getModelLocation();
550 uniformProjection = shaderList[6].getProjectLocation();
551 uniformView = shaderList[6].getViewLocation();
552
553 float tamañoPiramide = 1.0f;
554 glm::vec3 escalaAleta = glm::vec3(tamañoPiramide, tamañoPiramide, tamañoPiramide);
555 float distanciaCentro = 1.0f + (tamañoPiramide / 2.0f);
556 float alturaV = -2.5f;
557
558 // Aleta Izquierda (-X)
559 glm::mat4 modelAletaIzq = glm::translate(rocketBase, glm::vec3(-distanciaCentro, alturaV, 0.0f));
560 modelAletaIzq = glm::rotate(modelAletaIzq, glm::radians(90.0f), glm::vec3(0.0f, 0.0f, 1.0f)); // Punta hacia -X
561 modelAletaIzq = glm::scale(modelAletaIzq, escalaAleta);
562 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelAletaIzq));
563 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
564 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(view));
565 meshList[4]-->RenderMesh();
566
567 // Aleta Derecha (+X)
568 glm::mat4 modelAletaDer = glm::translate(rocketBase, glm::vec3(distanciaCentro, alturaV, 0.0f));
569 modelAletaDer = glm::rotate(modelAletaDer, glm::radians(-90.0f), glm::vec3(0.0f, 0.0f, 1.0f)); // Punta hacia +X
570 modelAletaDer = glm::scale(modelAletaDer, escalaAleta);
571 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelAletaDer));
572 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
573 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(view));
574 meshList[4]-->RenderMesh();
575
576 // Aleta Frontal (+Z) - Apuntando directo a la cámara (Forma un cuadrado con una X)
577 glm::mat4 modelAletaFrente = glm::translate(rocketBase, glm::vec3(0.0f, alturaV, distanciaCentro));
578 modelAletaFrente = glm::rotate(modelAletaFrente, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f)); // Punta hacia +Z
579 modelAletaFrente = glm::scale(modelAletaFrente, escalaAleta);
580 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelAletaFrente));
581 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
582 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(view));
583 meshList[4]-->RenderMesh();
584
585 // Aleta Trasera (-Z) - Apuntando hacia atrás
586 glm::mat4 modelAletaAtras = glm::translate(rocketBase, glm::vec3(0.0f, alturaV, -distanciaCentro));
587 modelAletaAtras = glm::rotate(modelAletaAtras, glm::radians(-90.0f), glm::vec3(1.0f, 0.0f, 0.0f)); // Punta hacia -Z
588 modelAletaAtras = glm::scale(modelAletaAtras, escalaAleta);
589 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelAletaAtras));
590 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
591 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(view));
592 meshList[4]-->RenderMesh();
593

```

*Ilustración 6: Código de las pirámides cuadrangulares.*

```

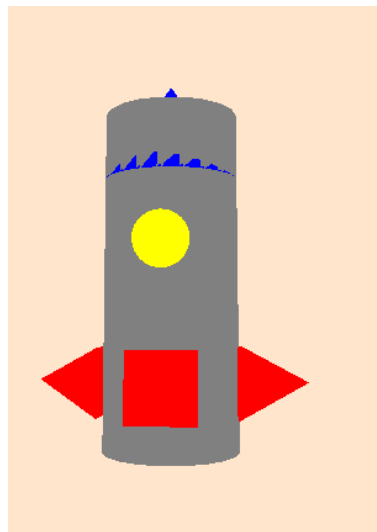
// Motor - cubo negro
shaderList[8].useShader(); // Negro
uniformModel = shaderList[8].getModelLocation();
uniformProjection = shaderList[8].getProjectLocation();
uniformView = shaderList[8].getViewLocation();

glm::vec3 escalaMotor = glm::vec3(0.6f, 0.6f, 0.6f);
float posicionMotorY = -2.6f;

glm::mat4 modelMotor = glm::translate(rocketBase, glm::vec3(0.0f, posicionMotorY, 0.0f));
modelMotor = glm::scale(modelMotor, escalaMotor);
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelMotor));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(view));
meshList[8]-->RenderMesh();

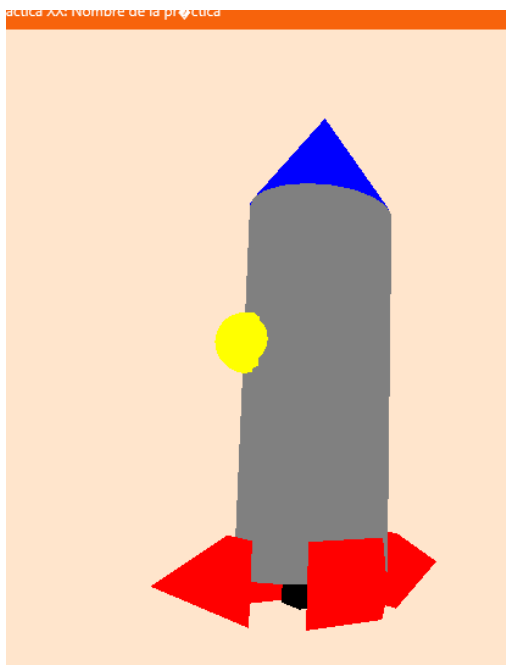
```

*Ilustración 7: Código del cubo negro.*

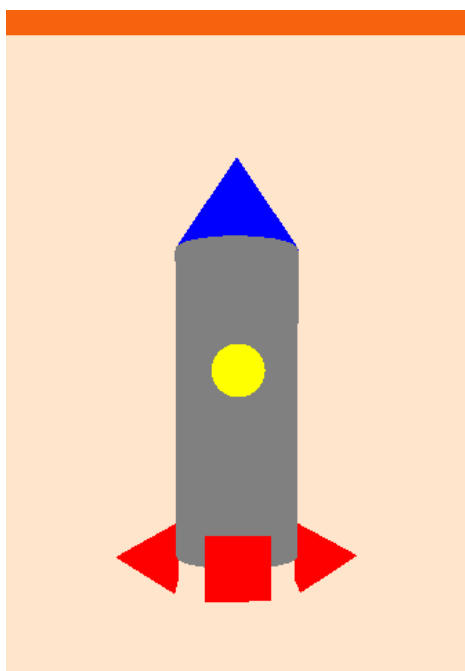


*Ilustración 8: Ejecución de prueba 1.*

Ahora que puedo rotar, moverme y hacer los movimientos para observar lo que construyo, me pareció más sencillo construir todo, más cuando yo diseñé desde cero la figura que deseaba construir. Es por ello que en las ilustraciones 9 y 10, se aprecia lo bien logrado de mi cohete, aunque también adopta la forma de la torre de un castillo.



*Ilustración 9: Primera ejecución del ejercicio 1 (cubo negro visible).*



*Ilustración 10: Segunda ejecución del ejercicio 1 (cubo negro no visible).*

## EJERCICIO 2

**Descripción:** Generar la figura de unión de 8 pirámides como la que se adjunta. Cada cara de cada pirámide es de un color diferente considerando que al ver una sola pirámide desde todos los ángulos se verían las caras triangulares de color: roja, verde, amarilla, magenta. Y la cara cuadrada se ve de color azul.

**Solución:** Como primer paso, agregué las dos funciones de la ilustración 11 para modificar el arreglo de vértices (VBO) con el fin de inyectar colores para cada cara de las pirámides cuadrangulares.

```
30 // Vamos agregar las funciones para poder colorear las caras de la pirámide cuadrangular
31 void CrearCaraTriangular()
32 {
33     unsigned int indices[] = { 0, 1, 2 };
34     GLfloat vertices[] = {
35         -0.5f, -0.5f, 0.5f, // Vértice inferior izquierdo
36         0.5f, -0.5f, 0.5f, // Vértice inferior derecho
37         0.0f, 0.5f, 0.0f // Punta superior
38     };
39     Mesh* caraTri = new Mesh();
40     caraTri->CreateMesh(vertices, indices, 9, 3);
41     meshList.push_back(caraTri);
42 }
43
44 void CrearCaraCuadrada()
45 {
46     unsigned int indices[] = { 0, 1, 2, 2, 3, 0 };
47     GLfloat vertices[] = {
48         -0.5f, -0.5f, 0.5f,
49         0.5f, -0.5f, 0.5f,
50         0.5f, -0.5f, -0.5f,
51         -0.5f, -0.5f, -0.5f
52     };
53     Mesh* caraCuad = new Mesh();
54     caraCuad->CreateMesh(vertices, indices, 12, 6);
55     meshList.push_back(caraCuad);
56 }
57
58
```

*Ilustración 11: Funciones para poder colorear correctamente las caras de las pirámides cuadrangulares.*

Y ya en el main inicializamos las funciones para que las podamos usar, justo como se observa en la ilustración 12.

```
392
393 int main()
394 {
395     mainWindow = Window(800, 600);
396     mainWindow.Initialise();
397     //Cilindro y cono reciben resolución (slices,
398
399     CrearCubo();//índice 0 en MeshList
400     CrearPiramideTriangular();//índice 1 en MeshL
401     CrearCilindro(20, 1.0f);//índice 2 en MeshLis
402     CrearCono(25, 2.0f);//índice 3 en MeshList
403     CrearPiramideCuadrangular();//índice 4 en Mes
404     CrearCaraTriangular();
405     CrearCaraCuadrada();
406     CreateShaders();

```

*Ilustración 12: Inicialización de las funciones creadas.*

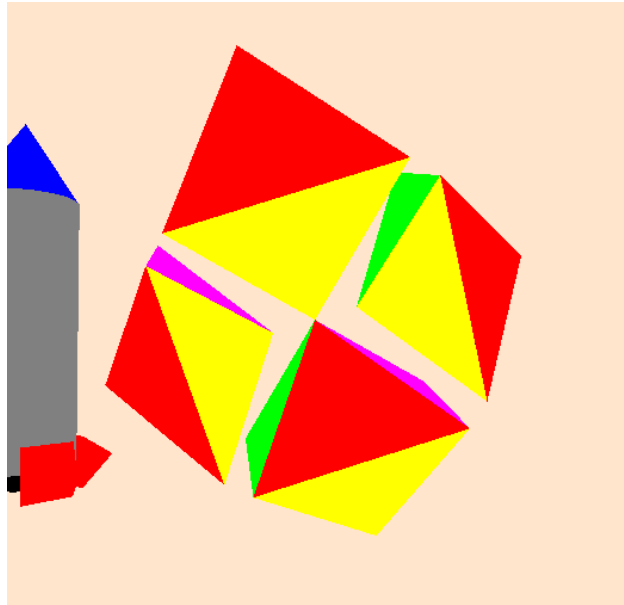
Como primera parte del código del ejercicio 2, como se ve en la ilustración 13, agregué variables, como en los ejercicios pasados, para poder editar las figuras de manera más sencilla. En esta ocasión lo dividí para los octaedros superior e inferior y para los octaedros izquierdo y derecho. De esta forma tenía más control para jugar con la perspectiva de las figuras. Como tenía duda de cómo hacer que los colores no se vean tan homogéneos, le pregunté a la IA cómo podía lograr este cambio para conseguir que cada pirámide tuviera un orden distinto de colores en sus caras laterales, dando como resultado la variable “giroColor” (ilustración 14). Después de todo, es una matriz bidimensional que asigna rotaciones locales en múltiplos de 90° (0, 90, 180, 270) a cada una de las 8 pirámides.

```

614 // Ejercicio 2: LAS 8 PIRÁMIDES CUADRANGULARES
615 glm::mat4 baseFigura = glm::translate(baseModel, glm::vec3(4.0f, 0.0f, 0.0f));
616
617 // --- ROTACIÓN GLOBAL DE LA FIGURA ---
618 float angGlobalX = 25.0f;
619 float angGlobalY = 45.0f;
620 baseFigura = glm::rotate(baseFigura, glm::radians(angGlobalX), glm::vec3(1.0f, 0.0f, 0.0f));
621 baseFigura = glm::rotate(baseFigura, glm::radians(angGlobalY), glm::vec3(0.0f, 1.0f, 0.0f));
622
623 // VARIABLES PARA EL PAR SUPERIOR E INFERIOR
624 glm::vec3 escalaArribaAbajo = glm::vec3(2.9f, 1.5f, 2.8f);
625 float pos_Y_Arriba = 1.5f;
626 float pos_Y_Abajo = -1.5f;
627
628 // VARIABLES PARA EL PAR IZQUIERDO Y DERECHO
629 glm::vec3 escalaIzqDer = glm::vec3(2.95f, 1.5f, 2.8f);
630 float pos_X_Der = 1.5f;
631 float pos_X_Izq = -1.5f;
632
633 // Arreglos de posición, escala y rotaciones estructurales
634 glm::vec3 posiciones[4] = {
635     glm::vec3(0.0f, pos_Y_Arriba, 0.0f), // 0: Arriba
636     glm::vec3(0.0f, pos_Y_Abajo, 0.0f), // 1: Abajo
637     glm::vec3(pos_X_Der, 0.0f, 0.0f), // 2: Derecha
638     glm::vec3(pos_X_Izq, 0.0f, 0.0f) // 3: Izquierda
639 };
640 glm::vec3 escalas[4] = { escalaArribaAbajo, escalaArribaAbajo, escalaIzqDer, escalaIzqDer };
641 float rotacionZ[4] = { 0.0f, 0.0f, -90.0f, 90.0f };
642 float rotacionY[4] = { 45.0f, 45.0f, 0.0f, 0.0f };
643
644 // Rota cada pirámide sobre su propio eje para mezclar los colores.
645 // Valores posibles para no deformar: 0.0f, 90.0f, 180.0f, 270.0f
646 float giroColor[4][2] = {
647     { 0.0f, 90.0f }, // Arriba: (Pirámide exterior, Pirámide interior)
648     { 90.0f, 180.0f }, // Abajo: (Pirámide exterior, Pirámide interior)
649     { 180.0f, 270.0f }, // Derecha: (Pirámide exterior, Pirámide interior)
650     { 270.0f, 0.0f } // Izquierda: (Pirámide exterior, Pirámide interior)
651 };
652
653
654

```

Ilustración 13: Parte 1 del código del ejercicio 2.



*Ilustración 14: Primera versión de la ejecución del ejercicio 2.*

Y ahora tenemos dos ciclos for en la ilustración 15, el primero recorre los arreglos estructurales para calcular la matriz base (`modelGrupo`) de las 4 posiciones espaciales. Y el segundo dibuja dos pirámides por cada rama del bucle principal: la primera desplazada hacia afuera ( $0.5f$ ) y la segunda desplazada hacia adentro ( $-0.5f$ ) e invertida ( $180.0f$  en X). Para lograr el coloreado independiente, prescinde de dibujar el objeto 3D completo. En su lugar, aplica transformaciones matriciales individuales para dibujar 4 mallas triangulares rotadas ( $0^\circ$ ,  $-90^\circ$ ,  $180^\circ$ ,  $90^\circ$ ) y 1 malla cuadrada para la base, inyectando a cada instrucción de dibujado el shader de color específico (Rojo, Verde, Amarillo, Magenta y Azul) antes de llamar a `RenderMesh()`.

```

654 // --- DIBUJAR LOS 4 GRUPOS ---
655 for (int grupo = 0; grupo < 4; grupo++) {
656     glm::mat4 modelGrupo = glm::translate(baseFigura, posiciones[grupo]);
657     modelGrupo = glm::rotate(modelGrupo, glm::radians(rotacionZ[grupo]), glm::vec3(0.0f, 0.0f, 1.0f));
658     modelGrupo = glm::rotate(modelGrupo, glm::radians(rotacionY[grupo]), glm::vec3(0.0f, 1.0f, 0.0f));
659     modelGrupo = glm::scale(modelGrupo, escalas[grupo]);
660
661     // --- DIBUJAR LAS 2 PIRÁMIDES DE CADA GRUPO ---
662     for (int pir = 0; pir < 2; pir++) {
663         glm::mat4 modelPiramide;
664         if (pir == 0) {
665             // Pirámide 1 (Apuntando hacia afuera)
666             modelPiramide = glm::translate(modelGrupo, glm::vec3(0.0f, 0.5f, 0.0f));
667         }
668         else {
669             // Pirámide 2 (Apuntando hacia adentro)
670             modelPiramide = glm::translate(modelGrupo, glm::vec3(0.0f, -0.5f, 0.0f));
671             modelPiramide = glm::rotate(modelPiramide, glm::radians(180.0f), glm::vec3(1.0f, 0.0f, 0.0f));
672         }
673
674         // APLICAMOS EL GIRO DE COLOR AQUÍ (Sin afectar la forma de la figura)
675         modelPiramide = glm::rotate(modelPiramide, glm::radians(giroColor[grupo][pir]), glm::vec3(0.0f, 1.0f, 0.0f));
676     }
677 }
678
679

```

*Ilustración 15: Parte 2 del código del ejercicio 2.*

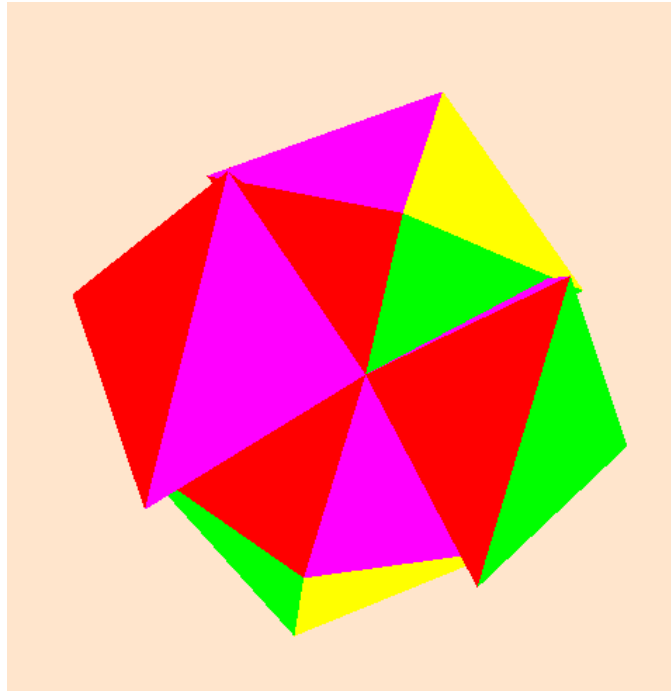
Para concluir, tenemos la ilustración 16 que construye la pirámide ensamblándola modularmente cara por cara. Dando como resultado la ilustración 17.

```

678 // 1. Cara Frontal (Roja - Shader 6)
679 shaderList[6].useShader();
680 glUniformMatrix4fv(shaderList[6].getModelLocation(), 1, GL_FALSE, glm::value_ptr(modelPiramide));
681 glUniformMatrix4fv(shaderList[6].getProjectLocation(), 1, GL_FALSE, glm::value_ptr(projection));
682 glUniformMatrix4fv(shaderList[6].getViewLocation(), 1, GL_FALSE, glm::value_ptr(view));
683 meshList[5]->RenderMesh();
684
685 // 2. Cara Derecha (Verde - Shader 5)
686 shaderList[5].useShader();
687 glm::mat4 mDer = glm::rotate(modelPiramide, glm::radians(-90.0f), glm::vec3(0.0f, 1.0f, 0.0f));
688 glUniformMatrix4fv(shaderList[5].getModelLocation(), 1, GL_FALSE, glm::value_ptr(mDer));
689 glUniformMatrix4fv(shaderList[5].getProjectLocation(), 1, GL_FALSE, glm::value_ptr(projection));
690 glUniformMatrix4fv(shaderList[5].getViewLocation(), 1, GL_FALSE, glm::value_ptr(view));
691 meshList[5]->RenderMesh();
692
693 // 3. Cara Trasera (Amarilla - Shader 7)
694 shaderList[7].useShader();
695 glm::mat4 mAtras = glm::rotate(modelPiramide, glm::radians(180.0f), glm::vec3(0.0f, 1.0f, 0.0f));
696 glUniformMatrix4fv(shaderList[7].getModelLocation(), 1, GL_FALSE, glm::value_ptr(mAtras));
697 glUniformMatrix4fv(shaderList[7].getProjectLocation(), 1, GL_FALSE, glm::value_ptr(projection));
698 glUniformMatrix4fv(shaderList[7].getViewLocation(), 1, GL_FALSE, glm::value_ptr(view));
699 meshList[5]->RenderMesh();
700
701 // 4. Cara Izquierda (Magenta - Shader 4)
702 shaderList[4].useShader();
703 glm::mat4 mIzq = glm::rotate(modelPiramide, glm::radians(90.0f), glm::vec3(0.0f, 1.0f, 0.0f));
704 glUniformMatrix4fv(shaderList[4].getModelLocation(), 1, GL_FALSE, glm::value_ptr(mIzq));
705 glUniformMatrix4fv(shaderList[4].getProjectLocation(), 1, GL_FALSE, glm::value_ptr(projection));
706 glUniformMatrix4fv(shaderList[4].getViewLocation(), 1, GL_FALSE, glm::value_ptr(view));
707 meshList[5]->RenderMesh();
708
709 // 5. Base Cuadrada (Azul - Shader 2)
710 shaderList[2].useShader();
711 glUniformMatrix4fv(shaderList[2].getModelLocation(), 1, GL_FALSE, glm::value_ptr(modelPiramide));
712 glUniformMatrix4fv(shaderList[2].getProjectLocation(), 1, GL_FALSE, glm::value_ptr(projection));
713 glUniformMatrix4fv(shaderList[2].getViewLocation(), 1, GL_FALSE, glm::value_ptr(view));
714 meshList[6]->RenderMesh();
715
716 }
717
718
719

```

*Ilustración 16: Parte 3 del código del ejercicio 2.*



*Ilustración 17: Ejecución del ejercicio 2.*

## CONCLUSIÓN

Al momento de construir el cohete, retomé la idea de que parametrizar el código con variables me permitía ajustar escalas y posiciones de múltiples objetos al mismo tiempo sin tener que reescribir todo, facilitando adaptar el resultado a mi boceto inicial. El mayor reto fue el segundo ejercicio, donde la interpretación geométrica de las pirámides cuadrangulares me causó confusión al principio. Sin embargo, descubrí que, al construir los objetos de forma modular, podía aplicar *shaders* de color independientes sin que se mezclaran o se deformara la malla. Además, usar arreglos bidimensionales para aplicar rotaciones locales me ayudó a desfasar los colores para que la figura final resaltara correctamente y no luciera como una plasta de color. En general, la implementación en código fue mucho más fluida al ordenar la lógica de las matrices y apoyarme en herramientas de IA para resolver problemas de rotación espacial que no tenía del todo claros.

## BIBLIOGRAFÍA

Google. (2026). *Gemini* [Modelo de lenguaje grande]. <https://gemini.google.com>